The *take-away*

Everything from the hackathon in one place: how to work with your AI, every step from warm-up to a live app, and every useful prompt and link. It's yours to keep.

Please read this first

This is a personal learning exercise. Use fictional or public sample content only. Do not enter, upload or share any LBG data in AI tools, GitHub repositories, prompts, screenshots or published websites. More on staying safe.

The latest copy is at hackday.bonhard.ai/talks/takeaway.pdf. Organised by Phil Bonhard. This guide does not claim endorsement by any organisation or approval of any tool.

Start here

How to use this

This is everything from the hackathon, written to be read on its own. You don't need the talks, and you can dip in anywhere.

Follow the five stops

The steps run in the same order as the My day board, from warm-up to a live app.

Copy the prompts

On the web page, each prompt has a Copy button. In the PDF, select the text. Change the words in [square brackets] to your own.

Read what comes back

A prompt is a starting point. Adjust the wording to your idea, then read and challenge what the AI gives you.

1. **Warm-up done**
   Hello world on your own computer
2. **GitHub connected**
   Your AI tool can save to your account
3. **Hello world pushed**
   Your page is on GitHub with a live address
4. **PRD shared**
   A short plan, reviewed and saved
5. **App live**
   Someone else can open it on their own device

Talk to it like a colleague

Think of your AI as a very fast colleague who's never met you. It knows a lot about building things and nothing about who you're designing for. That's your job.

You don't need special words. Brief it the way you'd brief a colleague. If the answer isn't right, say so. It won't be offended.

Ask it for help with anything, including "how do I...?"

- **Tell it who you are.** "I'm a designer. Explain things in plain words."
- **Give it context.** Who it's for, the problem, and what good looks like.
- **Keep talking.** Ask follow-up questions, and say "that's not what I meant".
- **Let it question you.** Ask it to ask you questions before it builds anything.

Show it, don’t describe it

A screenshot beats a paragraph. If something looks wrong, or you see an error you don’t understand, grab it, paste it and ask “what’s going on here?”

On a Mac

Grab part of the screen. Press `⌘ Cmd` `⇧ Shift` `4`, then drag a box round the bit you need.

Put it in the chat. Drag the thumbnail (bottom right) into the chat. Or add `Ctrl` to copy it, then press `⌘ Cmd` `V`.

On Windows

Grab part of the screen. Press `⊞ Win` `⇧ Shift` `S`, then drag a box round the bit you need.

Put it in the chat. It’s copied already. Click in the chat and press `Ctrl` `V`.

> **Grab only the bit you need.** A full-screen shot can catch Teams, email, notifications and other tabs. Share only a screenshot of your own sample product.

Stay in charge

AI can be confidently wrong. The cure is simple: small steps, and look at the result every time.

One change at a time

Ask, check, then the next thing.

Check it yourself

Open it, click it, try it on your phone. “Done” doesn’t always mean done.

Save when it works

Publish a working version before a big change.

If in doubt, ask it to explain its choices. “What were the options? Why this one?” That’s how you learn, and it’s often how you catch the mistake.

Going round in circles? Start a fresh chat and explain where you’ve got to.

Permissions, money and getting stuck

These tools can create files, run commands and use your accounts. That’s what makes them useful, and it’s why you read before you say yes.

Read before you say yes

Your AI asks permission before it does things. Say yes to your project folder and GitHub. Say no to anything you don’t understand, then ask for help.

Don’t buy anything on the spot

Asked to pay, upgrade or add a card without planning to? Stop and ask. Ran out of usage? Don’t top up in a panic.

The 10-minute rule

Stuck for ten minutes? Ask for help. Screenshot, paste, ask: your AI is the first helper you go to.

> **Never paste a password, key or token into a chat.** If your AI asks for one, say no. GitHub gives you a one-time code to type in yourself.

Choose your tool

Both do the same job. The warm-up takes about 25–35 minutes the first time (an estimate) and checks that your tools work. It's a rehearsal, not your product.

Claude

In the Claude desktop app's Code tab. Needs Claude Pro or higher, because Claude Code isn't on the free plan. Buying a plan is your choice.

Download the Claude app

Codex

In the ChatGPT desktop app. Listed on free and paid ChatGPT plans, with usage limits, so a paid plan gives you more room.

Download the ChatGPT app

> **Plans change.** Check the provider's own site before you sign up. Use your own personal accounts, not a work account or work email.

Make your first page

1. **Install the app and sign in.** In Claude, click the Code tab. In the ChatGPT app, choose Codex from the dropdown.
2. **Create a free GitHub account** at github.com/signup, with your own email. Your username is public. Remember how you signed up (email, Google or Apple) and always sign in the same way.
3. **Give it an empty folder.** Make a new folder called `hello-hackday` on your Desktop and open it in your tool.
4. **Ask for your page.** Paste this in. It may ask a question or two, or just make the page: either is fine. If it asks, answer in your own words, or say "you decide".

   > I'd like you to create a webpage that simply says hello world.

5. **See it.** Ask "Please open the page in my browser", or double-click the file ending in `.html`. The address starts with `file://`, which means it's only on your computer.

Connect your tool to GitHub

This lets your tool put your work on GitHub with one request. You won't type a password, and your tool won't open a browser. GitHub gives you a short one-time code, and you type it in yourself.

1. **Be signed in to GitHub somewhere,** in your usual browser, or on your phone if that's where you signed up.
2. **Paste this into the same chat.**

> Please connect this computer to my GitHub account, ready for the day. Go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Run gh auth login --hostname github.com --git-protocol https --web in the background, then show me the one-time code clearly. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token.
> 3. When I say done, run gh auth setup-git and gh auth status, and tell me my GitHub username.
> Don't create a repository or publish anything today.

3. **Type the code.** It looks like `ABCD-1234`. Go to github.com/login/device on any device where you're signed in, type it, press Continue, then Authorize. It works for about 15 minutes. Then tell your tool "done".
4. **You're finished when it tells you your GitHub username.** It's fine to approve git and gh, and installing them if it asks. Using Codex? Say yes when it asks to use the internet.

The loop you'll use all day

Everything from here is the same four moves. You'll go round them twice with your hello-world page, then all day with your real project.

1. **Change**
   Ask your AI
2. **Check**
   Look at it yourself
3. **Publish**
   Push to GitHub
4. **Share**
   Open it on a phone

Four words you'll hear

Repository (repo)

Your project's folder, stored on GitHub.

Commit

A save point you can go back to.

Push

Send your saved changes up to GitHub.

GitHub Pages

Turns your repository into a website with a real address.

Put your page online

Your page and its file become public on GitHub. Anyone with the address can see them. That's fine for a sample sentence and not for anything else.

Open your `hello-hackday` folder in your tool. Prefer your own words? Say you want to put this page on GitHub and make it a website, one step at a time. Or paste this.

> I'd like to publish this page on the web with GitHub Pages. It only contains fictional sample content. Please go one step at a time and explain each step in plain language.
> 1. Check that Git and the GitHub CLI (gh) are installed. If either is missing, tell me what you need to install and ask before you do it.
> 2. Check I'm signed in to GitHub with gh auth status. If I'm not, run gh auth login --hostname github.com --git-protocol https --web in the background and show me the one-time code. Don't open a browser or use any browser tools: I'll type the code myself at github.com/login/device. Never ask me for a password or token, and never store one.
> 3. Create a new PUBLIC repository called hello-hackday in my own GitHub account from this folder, commit the page and push it.
> 4. Turn on GitHub Pages for that repository, publishing from the main branch and the root folder.
> 5. Tell me my web address (it looks like https://USERNAME.github.io/hello-hackday/), check whether it is live yet, and tell me clearly if I need to wait.

Check it’s really there

Once your AI says it’s pushed, don’t take its word for it. Open GitHub and look.

1. **Read each request.** It’s fine to approve git and gh (GitHub’s own tool) for your `hello-hackday` folder and repository. If it asks for anything else, say no and ask for help.
2. **See your file on GitHub,** at `github.com/your-username/hello-hackday`. If you can see it, it’s on GitHub.
3. **Open your address on your phone.** It looks like `https://your-username.github.io/hello-hackday/`. Opening it on your phone proves someone else can open it too.

> **A 404 page usually means “not yet”.** GitHub can take up to 10 minutes the first time. Still missing after 15? Screenshot it and ask your AI to check that GitHub Pages is on for `hello-hackday`.

Change it and publish again

This is the moment it clicks: you changed something, checked it, and it's live for anyone.

1. **Ask for one change.** Reload the page on your own computer to check it.

   > Please add a smiley to my page and make it look a bit friendlier.

2. **Ask it to publish the change.**
3. **Refresh the live address.** Give it a minute. Still seeing the old version? Do a hard refresh: `Ctrl` `Shift` `R` on Windows, `⌘ Cmd` `Shift` `R` on a Mac.

> **That's the whole day in miniature:** connected, created, published, shared, changed, published again. Keep `hello-hackday` as your practice repository. Your real project gets a fresh start.

Pick something small

Choose a theme or bring your own idea. Every theme can be a simple website, small enough to finish in a day, and AI features inside your product are optional.

Personal website

A one-page site for a made-up person: who they are and how to get in touch.

Elf Help

A seven-day Elf on the Shelf planner for busy parents and carers.

Mortgage journey

A guided journey for a first-time buyer at a clearly made-up bank.

Your own idea

One user, one job and no more than three essential things it must do.

> **The briefs are on the site.** See the project themes, including two advanced routes for anyone who wants a challenge. Expect to work more on your own on those.

Start fresh

Your warm-up was a rehearsal. Leave it as it is, so your practice page stays safe and your real project stays tidy.

New folder

Somewhere easy to find, named after your project.

New repository

Its own home on GitHub, separate from `hello-hackday`.

New chat

A clean start for your AI, with your idea at the centre.

In Claude or Codex, open the new folder and paste this. Builders such as Lovable connect to GitHub in their own way, so follow their steps.

> I'm starting a new product project in this empty folder. It will only hold fictional or public sample content. Please guide me one step at a time, in plain language, to create a new public GitHub repository for it in my own account and connect this folder to it. Ask before installing anything, and never ask me for a password or token.

> **Working in a pair?** Both of you need to be able to open and edit the project. On GitHub, the owner can invite a partner from the repository's Settings → Collaborators. Swap the keyboard every 25–30 minutes.

Question your idea

Before you build, think the idea through with AI. Allow about 25 minutes for this and the next two steps. The goal is a small, clear plan that you understand and have agreed.

Replace [idea] with a sentence about your idea and paste this in. It works with any assistant, and you don't need to install anything. This part takes about 10 minutes.

> Help me think through this product before we build it. My idea is [idea]. Start with the intended user, their situation and the problem. Ask a few focused questions at a time; wait for my answers before asking questions that depend on them. Challenge vague claims and suggest alternatives, but let me make the decisions. Cover the main user journey, what makes the idea useful, assumptions we cannot yet prove, accessibility, the smallest useful release, what to leave out, and how we will test success. Keep the conversation within about ten minutes; record unresolved questions as assumptions instead of inventing answers. Then propose a concise PRD and ask me to review and agree it before building. Do not generate the implementation yet.

> **Answer in your own words.** If it challenges you, that's the point. Push back when you disagree, and say "I don't know yet" rather than inventing an answer. Unknowns become assumptions to test.

Choose the journey and scope

About 5 minutes. A small plan you can finish beats a big one you can't.

One user

One real-sounding person and the problem they have.

One complete task

Something they can do from start to finish.

Three features at most

Only the essential ones. Decide what you're leaving out, and say why.

> **Set the limits if it hasn't asked.** Tell it: "I need to publish today using [tool/host], with £0 additional service costs unless I explicitly choose otherwise. Use fictional data where needed."

Draft, challenge and save your PRD

A PRD is a short product plan. Let the AI draft it, then read it properly and fix anything that isn't what you meant. Allow about 10 minutes, and aim for one or two pages.

It should cover

- The user, their problem and the outcome you want
- Assumptions you still need to test
- The main user journey
- What's in the first release, and why
- What's left out
- Three to five checks that show it works
- How you'll publish it, and what comes next

Before you build, can you explain?

- **Who needs it, and why.**
- **One complete task,** step by step.
- **Why this first release:** what's in, what's out and the reason.
- **How you'll check it:** the checks from your PRD.

> **Save it as** `PRD.md` in your new project's GitHub repository, not the warm-up one. A list of features with no clear user need is a sign to go back a step.

Share your PRD

On the day, you send your PRD, your app and your product page to the organiser before the demos. It takes two minutes, and it puts your product on the big screen.

1. **Open** Share your PRD and add your name and your project's name. Working in a pair? Put both names.
2. **Add your addresses:** your app, and your product page if you've made one. Not live yet? Leave them blank and send it again later.
3. **Upload your PRD or link to it.** Is it still in a chat? Ask your AI: "Give me the PRD as a Markdown file I can download."

> **Your PRD is never published.** Sharing is open only around the event. After that, keep `PRD.md` in your repository: it's the best way to pick up where you left off.

Build in small slices

Get one piece working, check it against your PRD, then move on. Aim to get a first version online early, then improve it.

> Read the agreed PRD. Propose the smallest first slice we can build and publish now. Explain the files or services you need and any cost before adding dependencies. Implement in small steps, help me preview it and give me concrete checks after each step. Check each slice against the PRD and tell me if anything we learn should change it. Keep a working version before major changes. If a feature needs a backend, do not imply static hosting will run it. Use my chosen publishing route and help me publish early.

Update the plan

When building or testing teaches you something, change the PRD. That’s good design, not failure.

Polish isn’t proof

A good-looking result doesn’t show people need it. Testing tells you what to improve next.

Stop adding features

Leave time to fix what testing finds and to publish again.

Test it with someone else

Someone outside your pair tries the main task. It’s the quickest way to find out what you can’t see for yourself.

1. **Don’t coach.** Give them the address and ask them to attempt the main task.
2. **Watch where they hesitate,** and write down one thing you found.
3. **Tell your AI what happened.** Fill in [observation] and paste this.

> Here is what happened when someone tested my published product: [observation]. Compare it with the PRD acceptance checks. Suggest the smallest useful improvement, explain the trade-off and help me implement and verify it. Do not expand the scope unnecessarily.

4. **Publish again** and check the address on another device.

Make a product page

One simple web page about your product, on its own address, with a button to the real thing. It's the link you send when someone asks what you built. Allow about 15 minutes, after you've fixed the app.

1. **Ask your AI.** Give it your PRD (attach the file, or point it at `PRD.md`), and put your app's address in place of [address].

   > Make a simple, single web page about the product in my PRD. It is separate from the app itself. Include the product's name, who it's for, the problem it solves, what it does in three short points, and a clear button that opens the live app at [address]. Use only what's in the PRD: don't invent features, numbers, people or quotes. Keep it to one page that works well on a phone. Then help me publish it on its own address, one step at a time.

2. **Publish it on its own address,** the same way you published your hello-world page.
3. **Check it on another device,** and check the button opens your app.

The finish line

Six things to have by the end of the day. Check them before the demos.

- ✓ A live web address that someone else can open on their own device.
- ✓ Someone outside your pair has tested it, and you've written down one thing you found.
- ✓ You know how to change it and publish it again.
- ✓ A reviewed `PRD.md` saved in your project's GitHub repository.
- ✓ Everyone who built it can open the source or the editable project.
- ✓ You've chosen one next improvement.

> **What doesn't count.** A screenshot, a preview on your own computer or a link to a builder's editor isn't a published product: other people need to be able to open the live address. If it still isn't live by the end, say so honestly and write down what's left to fix. That's a useful result too.

Show what you made

About three minutes, in front of everyone. No slides needed: show the real thing.

1. **Who it's for** and the problem it solves.
2. **The live product,** opened at its address.
3. **What you did yourself.**
4. **One thing you learned.**
5. **The next improvement** you'd make.

> **Before you show it,** open the published address outside your builder and on another device, and run your PRD's checks. Note anything still incomplete. Working in a pair? Share the time, and each explain your own contribution.